

COAL

# SLIDE - 1

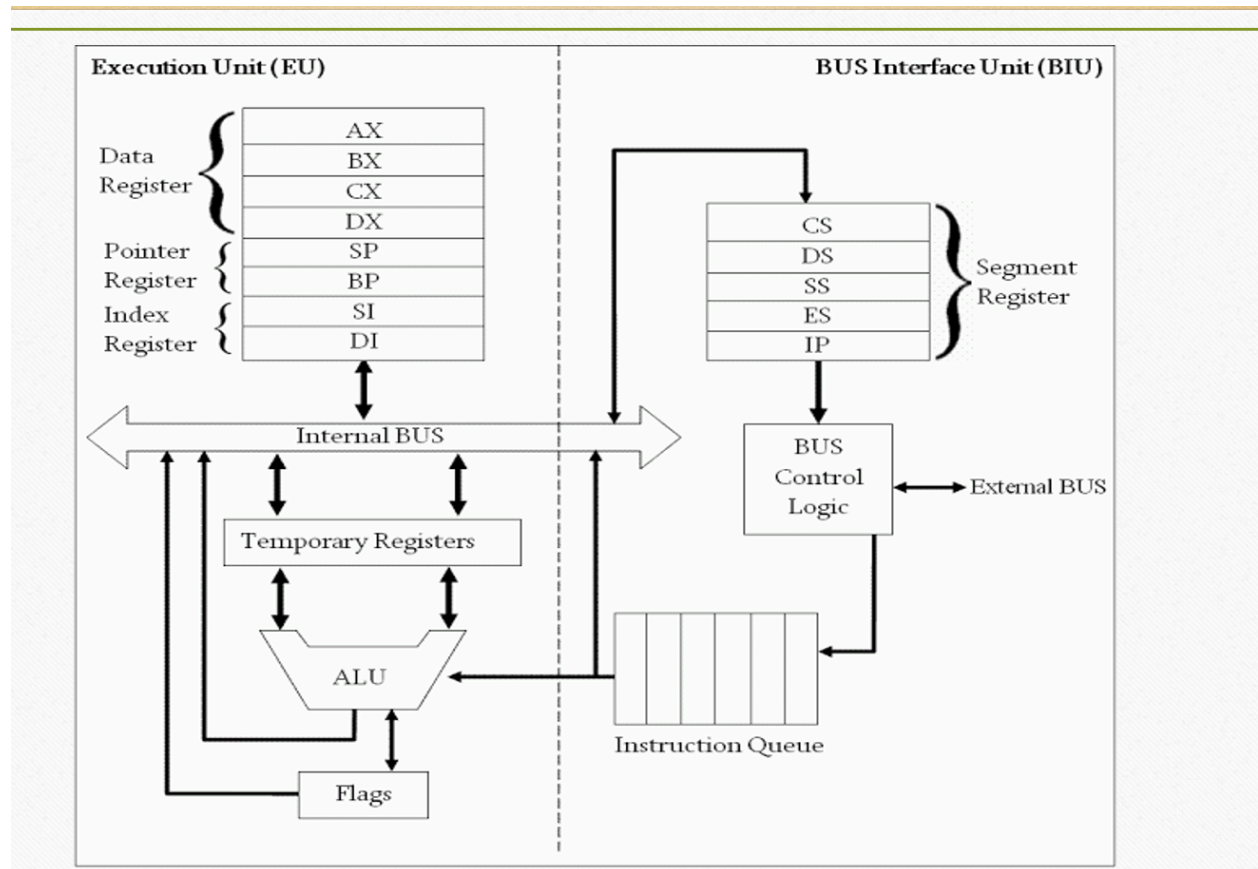
- **Instruction Set:** Instructions performed by CPU
- Two main components:
  - Execution Unit (EU) contain alu
  - Bus Interface Unit (BIU) • Facilitates communication between the EU and the memory or I/O circuits.
- Responsible for transmitting address, data and control signals on the buses.

## Address bus

- Carries the memory address from CPU to memory or I/O.
- For 8086: **20 bits** → can address  $2^{20} = 1,048,576$  bytes (1 MB).

cc

- **ALU:** arithmetic logic unit — does add, sub, logical ops.
- **Registers:** small fast storage inside CPU. We'll list them below.
- **Control Unit:** fetch-decode-execute cycle controller.
- **Flags/Status register:** shows result properties (zero, carry, sign, overflow, etc.)
- **Segment unit:** 8086 uses segment registers + offset to form physical address (we'll show conversion).



## General Registers (2 bytes each)

|    |    |    |             |              |
|----|----|----|-------------|--------------|
| AX | AH | AL | Accumulator | } Data group |
| BX | BH | BL | Base        |              |
| CX | CH | CL | Count       |              |
| DX | DH | DL | Data        |              |

|    |                     |                           |
|----|---------------------|---------------------------|
| SP | Stack pointer       | } Pointer and index group |
| BP | Base pointer        |                           |
| SI | Source index        |                           |
| DI | Destination index   |                           |
| IP | Instruction pointer |                           |

|                    |                    |                          |
|--------------------|--------------------|--------------------------|
| Flags <sub>H</sub> | Flags <sub>L</sub> | Status and control flags |
|--------------------|--------------------|--------------------------|

|    |       |                 |
|----|-------|-----------------|
| ES | Extra | } Segment group |
| CS | Code  |                 |
| DS | Data  |                 |
| SS | Stack |                 |

- **AX (Accumulator):** Used in arithmetic, logic and data transfer instructions. Also required in multiplication, division and input/output operations.
- **BX (Base):** It can hold a memory address that points to a variable.

- CX (Counter):** Act as a counter for repeating or looping instructions. These instructions automatically repeat and decrement CX and quit when equals to 0.

- DX (Data):** It has a special role in multiply and divide operations. Also used in input/output operations.

- Store addresses of instruction and data in memory.

- These values are used by the processor to access memory locations.

- CS (Code):** Defines the starting address of the section of memory holding code.

- DS (Data):** Defines the section of memory that holds most of the data used by programs.

- ES (Extra):** This is an additional data segment that is used by some of the string instructions.

- SS (Stack):** It defines the area of the memory used for stack

- IP - instruction pointer:** Always points to next instruction to be executed. IP register always works together with CS segment register.

- SI - source index register:** Can be used for pointer addressing of data. Works together with DS segment register.

- DI - destination index register:** Can be used for pointer addressing of data. Works together with ES segment register.

- BP - base pointer:** Primarily used to access parameters passed via the stack. Works together with SS segment register.

- SP – stack pointer:** Always points to top item on the stack. Works together with SS segment register.

## SLIDE - 2

- Converting Hexadecimal to Decimal

- Multiply each digit of the hexadecimal number from right to left with its corresponding power of 16.

- Converting Binary to Decimal

- Multiply each digit of the binary number from right to left with its corresponding power of 2.

- Converting Decimal to Binary

- Divide the decimal number by 2.

- Take the remainder and record it on the side.

- REPEAT UNTIL the decimal number cannot be divided into anymore.

- Converting Decimal to Hexadecimal

- Divide the decimal number by 16.

- Take the remainder and record it on the side.

- REPEAT UNTIL the decimal number cannot be divided into anymore.

Signed num bits

**Minimum value**

$$-2^{(n-1)}$$

**Maximum value**

$$+2^{(n-1)} - 1$$

For an **n-bit unsigned number**:

**Minimum value**

$$0$$

**Maximum value**

$$2^n - 1$$

# SLIDE - 3

## Why exactly 64 KB?

Because:

- Segment registers (CS, DS, SS, ES) are **16-bit**
  - A 16-bit register can hold values from **0000h to FFFFh**
  - That is  $2^{16} = 65,536 \text{ bytes} = 64 \text{ KB}$
- 
- Within the computer, each instruction is represented by a sequence of bits.
  - 16 bits instruction
  - 4 bit opcode, 6 bit operand 1, 6 bit operand 2
  - 4 bit opcode, 12 bit operand
  - 32 bits instruction
  - 64 bits instruction



## Rules

- Both source and destination operands should be the same size.
- Both operands cannot be segment registers simultaneously
- MOV DS, SS
- Using two segment registers as operands in an operation could create ambiguity about which segment of memory is being referenced.
- The destination operand cannot be a CS or an IP register
- MOV CS, AX
- Changing the CS register directly could disrupt the flow of program execution
- If the destination operand is a segment register, the source operand cannot be an immediate (constant) value. Instead, copy it from some other register like AX.
- MOV ES, [num1] is not valid.
- **No direct memory-to-memory moves.**
- mov [var1], [var2]
- Use register as intermediate:  
mov al, [var1] and mov [var2], al

## MODES

- immediate addressing mode: `mov ax, 0004`
- direct addressing mode: `mov ax, [0004]`
- register addressing mode: `mov ax, bx`
- register indirect addressing mode: `mov ax, [bx]`
- indexed addressing mode: `mov ax, [si + 04h]` :  
(Index + Displacement)
- Based + Indexed Addressing: `MOV AX, [BX + SI + 04h]`

## SLIDE - 4

status flags

- Set to 1 or cleared to 0 after execution of each instruction
- Sign flag (SF)
- Zero flag (ZF)
- Auxiliary carry flag (ACF)
- This flag is used in BCD (Binary-coded Decimal) operations.

Indicates whether a carry occurred from the lower nibble (the least significant 4 bits) to the upper nibble during an arithmetic operation

- Parity flag (PF)
- Overflow flag (OF)
- Carry flag (CF)

## control flags

- Dictate how the operations will be performed by the processor
  - How certain instructions are executed
  - Interrupt flag (IF)
  - Trap Flag (TF)
  - Directional flag (DF)
- 
- Determines the direction in which string operations will be processed in memory
  - $DF = 0$  (Forward Direction):
    - String operations will increment the index registers
    - Data is processed from lower memory addresses to higher memory addresses
  - $DF = 1$  (Backward Direction):
    - String operations will decrement the index registers.
    - This means data is processed from higher memory addresses to lower memory addresses.
  - CLD and STD
- 
- Enables single-step debugging by generating an interrupt after each instruction execution.
  - **TF = 1 (Trap Mode):**
    - When set, it allows a debugger to take control after each instruction, facilitating step-by-step execution.
  - **TF = 0 (Normal Mode):**

- When cleared, the CPU executes instructions normally without generating interrupts after each instruction.
- Controls whether the computer can respond to important signals (interrupts) from peripheral devices, like the keyboard or mouse
- **IF = 1** (Interrupts Enabled):
  - If you're typing in Word, the computer registers each keypress instantly because interrupts are enabled.
- **IF = 0** (Interrupts Disabled):
  - The CPU **ignores** outside signals temporarily.
  - During an update, interrupts might be disabled to ensure the update isn't interrupted by other tasks.

## SLIDE - 5

jz/je zf=1

jc array cf=1

js ign sf=1

jo verflow of=1

jp arity pf=1

unsign

ja bove cf=0&zf=0

jae cf=0

jb elow cf=1

jbe cf=1 || zf=1

signed

jg reater zf=0 &sf=of

jge sf==of

jl lesser sf!=of

jle zf=1 || sf!=of

## SLIDE - 6

Stack to • To reverse an input

## SLIDE - 7

### Comparison of Methods

---

| Method    | Use Case           | Advantages                   | Limitations                 |
|-----------|--------------------|------------------------------|-----------------------------|
| Registers | Simple values      | Fast and efficient           | Limited capacity            |
| Stack     | Nested calls       | Flexible, supports recursion | Slower than registers       |
| Memory    | Large/complex data | No size limits               | Slower due to memory access |